

Software Architecture Assignment 5: Component and Sequence Diagrams with Architectural Justification

1. Introduction and Case Summary

Selected System

The selected system is a Geolocation-Based Service Worker Marketplace Platform, continued consistently from Assignments 1–4.

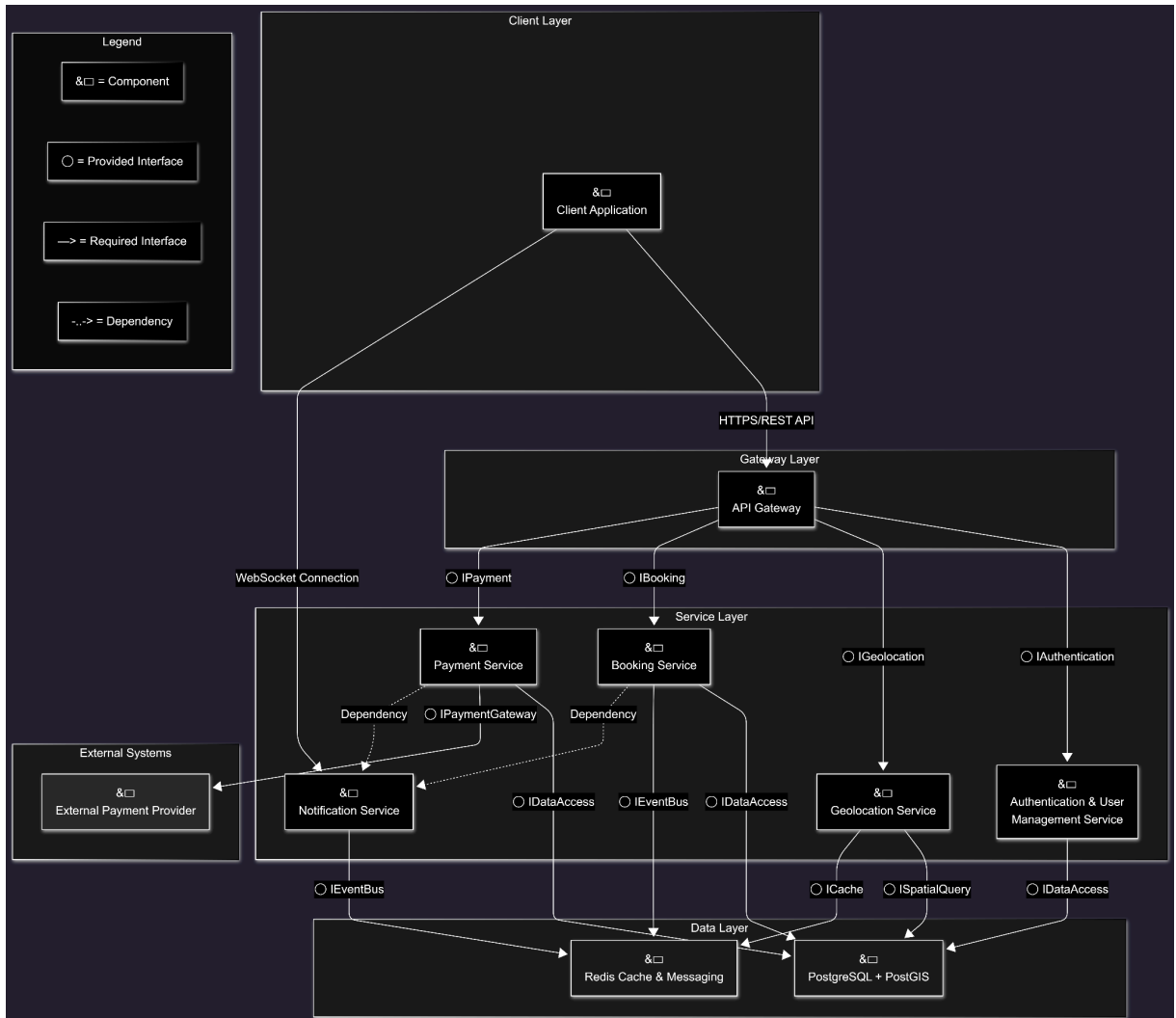
System Purpose and Main Users

The platform connects customers who require immediate or scheduled services (such as plumbing, electrical work, cleaning, and maintenance) with nearby service workers using real-time geolocation. Customers can post service requests, discover and compare nearby workers, communicate in-app, track job progress, and complete secure payments. Service workers manage profiles, browse and bid on jobs, receive real-time notifications, and track earnings. Administrators oversee verification, dispute resolution, analytics, and overall system health.

The system is designed as a scalable, highly available microservices-based platform, supporting high concurrency, low latency, and strict security and compliance requirements, as defined in Assignments 1 and 2 and justified through ADRs in Assignment 4.

2. Component Diagram

2.1 Component Diagram (Architectural View)



Architectural Components:

- API Gateway
- Authentication & User Management Service
- Geolocation Service
- Booking Service
- Payment Service
- Notification Service
- PostgreSQL with PostGIS
- Redis (Cache & Messaging)
- External Payment Provider

2.2 Component Diagram Description

The component diagram presents a high-level structural view of the system, reflecting the microservices architecture selected in ADR-1 (Architectural Style Selection). Each component represents an independently deployable service with a well-defined responsibility.

Component Responsibilities:

- **API Gateway:** Serves as the single entry point for all client requests. It handles request routing, authentication verification, rate limiting, and basic request validation.
- **Authentication & User Management Service:** Manages user registration, authentication (OAuth 2.0), authorization, roles (customer, service worker, administrator), and user profiles.
- **Geolocation Service:** Processes real-time location updates from service workers and performs geospatial queries to identify nearby workers using PostGIS.
- **Booking Service:** Manages service requests, job lifecycle states, worker assignment, and job status updates.
- **Payment Service:** Handles payment initiation, validation, transaction processing, and stores payment records while integrating with an external payment provider.
- **Notification Service:** Responsible for delivering real-time notifications to users via WebSockets and asynchronous messaging mechanisms.
- **PostgreSQL with PostGIS:** Stores transactional data and spatial data, ensuring ACID compliance and accurate geospatial queries.
- **Redis:** Used for caching frequently accessed data and for asynchronous messaging (Pub/Sub) to support real-time updates.
- **External Payment Provider:** Third-party system responsible for executing secure financial transactions.

Communication Patterns:

- Synchronous communication is implemented using REST APIs between the API Gateway and backend services.
- Asynchronous communication is used for notifications and event propagation via Redis Pub/Sub, following ADR-6 (Integration and Communication Strategy).

2.3 Architectural Justification

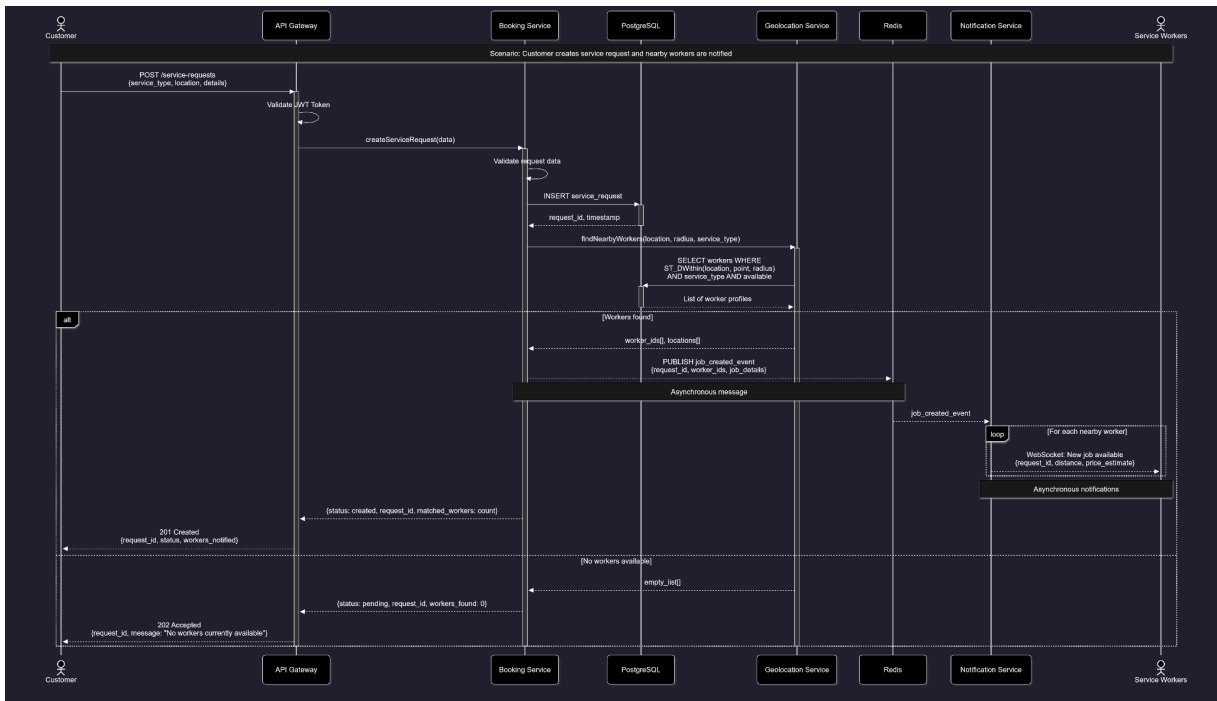
The decomposition of the system into independent services is driven by functional requirements defined in Assignment 1, such as user management, real-time worker discovery, booking management, and secure payment processing. Separating these concerns improves modularity and enables independent development and deployment.

Non-functional requirements and quality attributes identified in Assignment 2 strongly influenced this structure. Scalability and performance requirements justify independent scaling of high-load services such as the Geolocation and Booking services. Security requirements, including GDPR and PCI-DSS compliance, justify isolating the Payment Service and enforcing strict access boundaries.

This component structure directly follows architectural decisions documented in Assignment 4, particularly ADR-1 (Microservices Architecture), ADR-4 (PostgreSQL with PostGIS), and ADR-6 (Hybrid Communication Strategy), ensuring architectural consistency and traceability across assignments.

3. Sequence Diagrams

3.1 Sequence Diagram 1: Service Request Creation and Worker Matching



Description

Scenario: A customer creates a new service request, and the system identifies and notifies nearby available service workers.

Trigger: Customer submits a service request via the client application.

Interaction Flow:

1. The Customer sends a service request to the API Gateway.
2. The API Gateway forwards the request to the Booking Service.
3. The Booking Service validates the request and stores it in PostgreSQL.
4. The Booking Service requests nearby available workers from the Geolocation Service.
5. The Geolocation Service performs a spatial query using PostGIS.
6. The Geolocation Service returns a list of suitable workers.
7. The Booking Service publishes a job-created event to Redis.
8. The Notification Service consumes the event and notifies workers asynchronously.

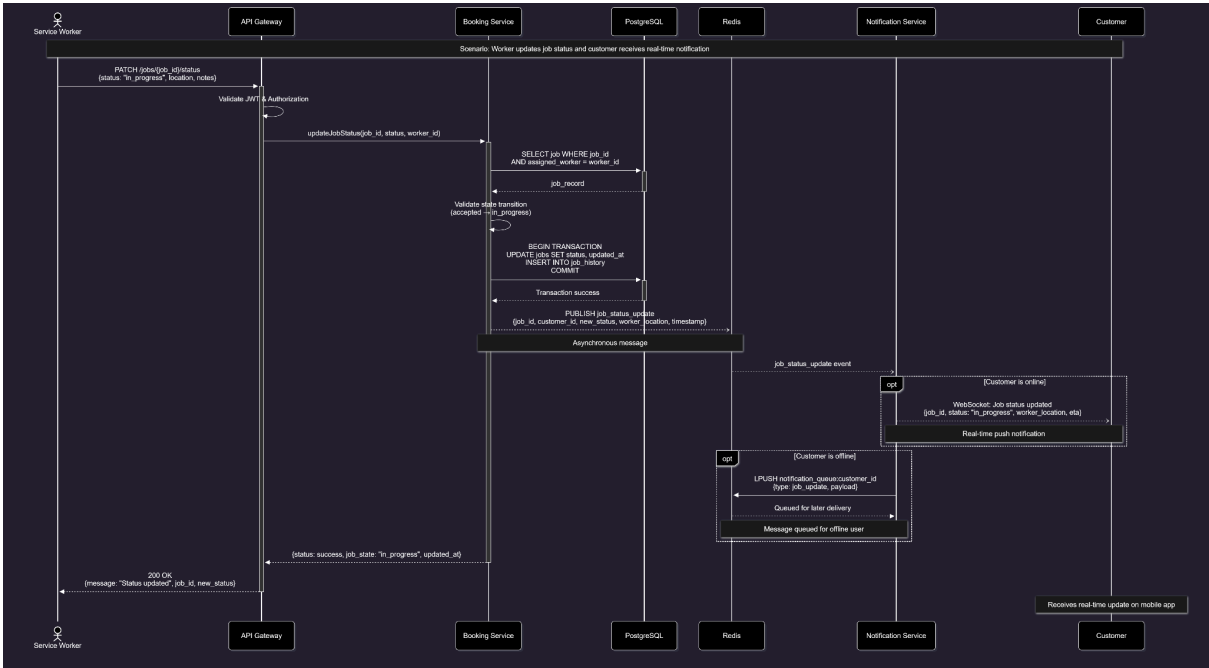
alt: If no workers are available, the Booking Service notifies the customer about the unavailability.

Architectural Justification

This sequence is derived from functional requirements related to service request creation and real-time worker discovery (Assignment 1). Performance efficiency is a prioritized quality attribute (Assignment 2), requiring fast response times and non-blocking notification delivery.

Asynchronous messaging via Redis ensures scalability and responsiveness under high load, in accordance with **ADR-6**. The interaction structure supports performance and availability while minimizing coupling between services.

3.2 Sequence Diagram 2: Real-Time Job Status Update



Description

Scenario: A service worker updates the status of an ongoing job, and the customer receives real-time updates.

Trigger: Service worker submits a job status update.

Interaction Flow:

1. The Service Worker sends a status update to the API Gateway.
2. The API Gateway forwards the update to the Booking Service.
3. The Booking Service updates the job state in PostgreSQL.
4. The Booking Service emits a job-update event.
5. The Notification Service delivers the update to the customer via WebSocket.

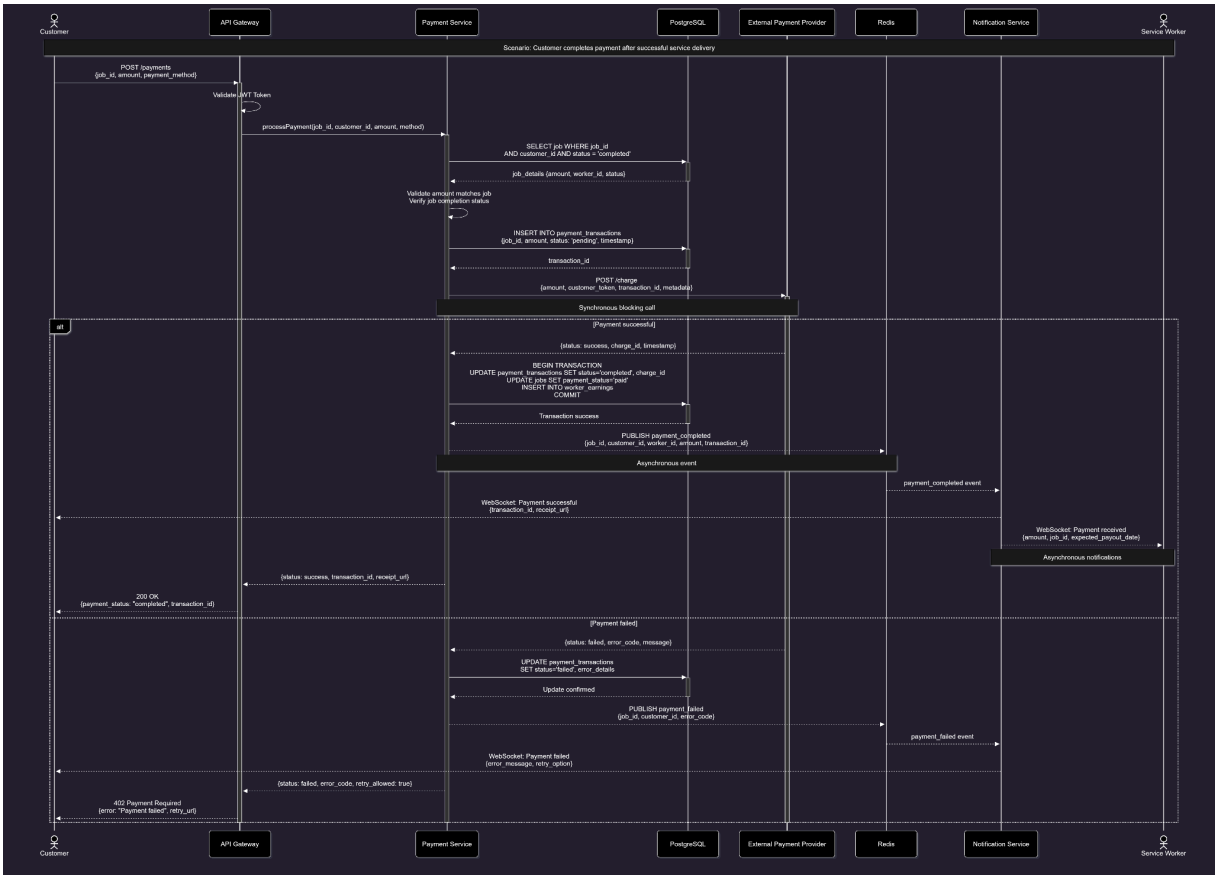
opt: If the customer is offline, the notification is queued for later delivery.

Architectural Justification

This scenario supports functional requirements for job tracking and real-time communication (Assignment 1). Reliability and performance efficiency are critical quality attributes (Assignment 2).

WebSocket-based communication enables low-latency updates, while persistent storage ensures consistency and recoverability. This design aligns with **ADR-6**, which specifies hybrid synchronous and asynchronous communication strategies.

3.3 Sequence Diagram 3: Secure Payment Processing



Description

Scenario: A customer completes payment after a service has been successfully delivered.

Trigger: Customer initiates payment.

Interaction Flow:

1. The Customer sends a payment request to the API Gateway.
2. The API Gateway routes the request to the Payment Service.
3. The Payment Service validates the payment request.
4. The Payment Service communicates with the External Payment Provider.
5. The External Payment Provider processes the transaction and returns the result.
6. The Payment Service stores the transaction record in PostgreSQL.
7. The Notification Service informs both customer and worker about payment status.

alt: If the payment fails, the customer is notified and offered a retry option.

Architectural Justification

This sequence is based on functional requirements for secure payment processing (Assignment 1) and security quality attributes (Assignment 2). Isolation of the Payment Service minimizes security risks and supports PCI-DSS compliance.

Synchronous communication is used for payment confirmation to ensure transactional integrity, while notifications are delivered asynchronously to maintain responsiveness. This design follows ADR-4 (Database Selection) and ADR-6 (Communication Strategy).

4. Consistency and Traceability

All components appearing in the sequence diagrams are defined in the component diagram. Interactions strictly adhere to the architectural decisions documented in Assignment 4. For example, REST is used for synchronous request-response flows, while Redis-based messaging supports asynchronous communication.

Each interaction can be traced back to functional requirements from Assignment 1 and quality attributes prioritized in Assignment 2, ensuring architectural consistency across all assignments.

5. Assumptions and Open Questions

Assumptions:

- Redis provides sufficient performance and reliability for real-time messaging at current scale.
- External payment providers remain available and compliant with regulatory requirements.
- Traffic growth follows projected patterns.

Open Questions:

- Should premium users receive prioritized worker matching or higher quality-of-service guarantees?
- At what scale should Redis Pub/Sub be replaced with a dedicated event-streaming platform such as Kafka?
- How will future regulatory changes impact data retention and auditing requirements?